

[T Text & String Manipulation](#)

INSTANT

[📅 Date & Time Arithmetic](#)[🌟 Data Analysis Diploma • Session 26](#)

T-SQL

BUILT-IN FUNCTIONS

Scalar Functions · Conversion Functions · CASE WHEN — From Excel Thinking to Database Thinking

3 Parts**Core Foundations**

Scalar, Cast/Convert & CASE

31 Slides**Interactive Guide**

Clean, Convert & Categorize

6 Tables**Academy Database**

Real Business Queries

Start Session 26 →[🔄 CAST & CONVERT Data Types](#)[🔗 CASE WHEN Decision Logic](#)

— WHAT WE WILL COVER IN THE NEXT 3 HOURS

Session Roadmap

What we will cover in the next 3 hours to transform, convert, and logically classify data:

1 Scalar Functions

~60 min

Transform one value at a time — text, dates, numbers.

2 Conversion Functions

40 min

Move data safely between types with CAST & CONVERT.

3 CASE WHEN

~50 min

Build IF-style logic directly inside a query.

🔗 BY THE END OF THIS SESSION, YOU WILL BE ABLE TO:

📄 Clean and reshape text, dates and numbers with scalar functions

🔄 Convert data between types safely using CAST and CONVERT

🔗 Write CASE WHEN logic to categorize and label rows

✅ Apply all three skills together on real business tables

💡 Interactive roadmap: Click each track to inspect syllabus coverage. Real operations dataset used across all 3 modules.

— DATABASE QUICK REFRESHER

Database Quick Refresher

Today's examples use the Academy's own operations database (Students, Courses, Payments, Campaigns, Leads, SalesAgents):

Students	ACADEMY	Courses	CATALOG	Payments	FINANCE	Campaigns	MARKETING	Leads	PIPELINE	SalesAgents	STAFF
StudentID PK		CourseID PK		PaymentID PK		CampaignID PK		LeadID PK		AgentID PK	
StudentName		CourseName		EnrollmentID FK		CampaignName		CampaignID FK		AgentName	
Email		Category		Amount		Channel		LeadName		Commission	
City		Price		PaymentDate		StartDate		Status		MonthlyTarget	
EnrollDate		DurationHrs		Method		EndDate		CreatedDate			
Status											

KEY IDENTIFIERS: ● Primary Key (PK) ● Foreign Key (FK)

Click any card to inspect columns



Good news: You already used SELECT, WHERE, and JOIN with these tables in earlier sessions. Today we make the values inside each column smarter — cleaning text, converting types, and labelling rows with logic.



PART 1 OF 3 • SCALAR FUNCTIONS

Scalar Functions

Transform text, dates, and numbers — **one row at a time**

— WHAT IS A SCALAR FUNCTION?

What is a Scalar Function?

A function that takes one value in, and returns exactly one value out — for every single row

A function that takes **one value in**, and returns **exactly one value out** — for every row.

INPUT COLUMN VALUE

"john doe"



SCALAR FUNCTION

UPPER()



OUTPUT VALUE

"JOHN DOE"

Σ Scalar Function

Acts on **ONE** value per row. Returns one output for every input row — the row count never changes.

```
SELECT UPPER(StudentName)
FROM Students;
```

500 rows in → 500 rows out

Aggregate Function

Acts on **MANY** rows at once. Collapses an entire group of rows into a single summary value.

```
SELECT COUNT(*)
FROM Students;
```

500 rows in → 1 row out

— STRING FUNCTIONS — THE ESSENTIALS

String Functions — The Essentials

Clean up messy text values before they reach an analytics report or executive dashboard

 **Interactive Tester:**

Test
string:

John Doe

Reset

UPPER()

Converts text to ALL CAPS

UPPER('john') → 'JOHN'

Live Output:

JOHN DOE

LOWER()

Converts text to all lowercase

LOWER('JOHN') → 'john'

Live Output:

john doe

LEN()

Counts the number of characters

LEN('John') → 4

Live Output:

12

TRIM()

Removes leading & trailing spaces

TRIM(' John ') → 'John'

Live Output:

'John Doe'

 **Excel habit to unlearn:** In Excel you nest `=TRIM(UPPER(A1))` inside a single cell. In SQL, functions wrap the same way — `UPPER(TRIM(StudentName))` — but run once for every row automatically across the entire table!

— LIVE DEMO — CLEANING STUDENT DATA

Live Demo — Cleaning Student Data

Standardize inconsistent StudentName and Email entries in the Students table

</> T-SQL Query

Students Table

```
SELECT
    StudentID,
    TRIM(StudentName)      AS CleanName,
    LOWER(TRIM(Email))     AS CleanEmail,
    LEN(TRIM(StudentName)) AS NameLength
FROM Students
WHERE LEN(StudentName) <> LEN(TRIM(StudentName));
```

● Executes text sanitization and length comparison in a single pass

✓ TRIM(StudentName)

Removes accidental leading and trailing spaces typed during student enrollment.

✓ LOWER(TRIM(Email))

Standardizes email casing to ensure reliable lookups, joins, and deduplication.

🔍 Smart WHERE Filter

LEN <> LEN(TRIM) isolates only the dirty records needing cleanup!

📄 Filtered Query Output (Dirty Records)

2 rows affected

STUDENTID	CLEANNAME	CLEANEMAIL	NAMELENGTH
1042	Mona Farid	mona.farid@mail.com	10
1078	Karim Adel	karim.adel@mail.com	10

— EXTRACT, JOIN & REPLACE

String Functions — Extract, Join & Replace

Pull pieces out of text, glue values together, and swap old text for new

</> PRODUCTION SQL EXAMPLE

```
SELECT
  StudentName,
  Email,
  SUBSTRING(Email, CHARINDEX('@', Email) + 1, 50) AS EmailDomain,
  CONCAT(StudentName, ' <', Email, '>') AS ContactLabel,
  REPLACE(Email, 'gmail.com', 'inbox.com') AS UpdatedEmail
FROM Students;
```

SUBSTRING(text, start, length)

Extracts part of a text value, starting at a specific position (1-indexed in SQL).

`SUBSTRING('Data Analytics', 1, 4) → 'Data'`

CONCAT(a, b, c, ...)

Joins two or more values into one text string (handles NULL values cleanly without crashing).

`CONCAT('User: ', 'Ali') → 'User: Ali'`

REPLACE(text, old, new)

Swaps every occurrence of old target text with new replacement text.

`REPLACE('mail.com', 'mail', 'inbox') → 'i...`

 **CHARINDEX('@', Email):** Returns the character index of the @ symbol so SUBSTRING knows dynamically where to cut, regardless of username length!

DOMAINS & MAILING LABELS

Live Demo — Domains & Mailing Labels

Build a marketing-ready label and check which email domains students use

</> Domain Extraction & Marketing Labels

ORDER BY Domain

```
SELECT
    StudentName,
    City,
    SUBSTRING(Email, CHARINDEX('@', Email) + 1, 50) AS Domain,
    CONCAT(StudentName, ' - ', City) AS MailingLabel
FROM Students
ORDER BY Domain;
```

● Calculates substring dynamic offset using CHARINDEX position

Output Preview

2 rows

STUDENT	CITY	DOMAIN	MAILINGLABEL
Mona Farid	Cairo	mail.com	Mona Farid - Cairo
Karim Adel	Giza	gmail.com	Karim Adel - Giza



CHARINDEX finds the position of '@' so **SUBSTRING** knows exactly where the domain begins — no matter how long the name before it is!

— DATE FUNCTIONS — WORKING WITH TIME

Date Functions — Working with Time

Calculate ages, deadlines, and durations directly inside SQL Server

T-SQL DATE ARITHMETIC EXAMPLES

```
SELECT
    GETDATE()                AS TodayDateTime,
    DATEADD(DAY, 30, GETDATE()) AS ThirtyDaysFromNow,
    DATEDIFF(DAY, EnrollDate, GETDATE()) AS DaysSinceEnrollment
FROM Students;
```

GETDATE()

Returns the SQL Server's current date and time — no arguments needed.

```
GETDATE() → 2026-09-23 17:45:00
```

DATEADD(unit, n, date)

Adds (or subtracts, with negative n) a specified date interval to a date.

```
DATEADD(DAY, 30, GETDATE())
```

DATEDIFF(unit, start, end)

Counts how many units (days, months, years) lie between start and end date.

```
DATEDIFF(DAY, EnrollDate, GETDATE())
```

 **Common DATEADD/DATEDIFF units:** DAY, WEEK, MONTH, YEAR, HOUR, MINUTE — always written as the first argument, **unquoted**.

— LIVE DEMO — CAMPAIGN DURATION

Live Demo — Campaign Duration

Measure how long each marketing campaign ran, and how long ago it ended

</> Campaign Lifespan Analytics

Campaigns Table

```
SELECT
    CampaignName,
    StartDate,
    EndDate,
    DATEDIFF(DAY, StartDate, EndDate)
        AS CampaignDurationDays,
    DATEDIFF(DAY, EndDate, GETDATE())
        AS DaysSinceCampaignEnded
FROM Campaigns;
```

● Calculates dual time windows with start and end dates

Campaign Metrics Output

2 campaigns

CAMPAIGN	START	END	DURATION	DAYSSINCEENDED
Summer Push	2026-03-01	2026-03-31	30	113
Back to School	2026-06-01	2026-06-25	24	27

🕒 **Insight:** Notice both calculations reuse `DATEDIFF` with the same unit (`DAY`) but different column pairs — one gives a measure of **length/duration**, the other a measure of **recency**.

— MATH FUNCTIONS — ROUNDING & PRECISION

Math Functions — Rounding & Precision

Control decimal places and handle positive/negative values cleanly



Interactive Numeric Tester:

Amount:

428.65

Reset

ROUND(n, digits)

Rounds to the given number of decimal places.

 $\text{ROUND}(428.65, 0) = 429$

ABS(n)

Removes the negative sign — returns the absolute magnitude.

 $\text{ABS}(428.65 - 500) = 71.35$

CEILING(n)

Rounds up to the nearest whole integer.

 $\text{CEILING}(428.65) = 429$

FLOOR(n)

Rounds down to the nearest whole integer.

 $\text{FLOOR}(428.65) = 428$


Excel parallel: `=ROUND(n, digits)` behaves exactly like Excel's `ROUND()`. `CEILING` and `FLOOR` directly match Excel's `ROUNDUP` and `ROUNDDOWN` when rounding to whole numbers.

— LIVE DEMO — COMMISSION SNAPSHOT

Live Demo — Commission Snapshot

Round agent commission values and calculate progress toward monthly target

</> Sales Agent Commission & Quota %

SalesAgents Table

```
SELECT
    AgentName,
    MonthlyTarget,
    Commission,
    ROUND(Commission, 2)
      AS RoundedCommission,
    ROUND(Commission / MonthlyTarget * 100, 1)
      AS CommissionPct
FROM SalesAgents;
```

● Multiplies and divides before executing outer ROUND function

Agent Calculation Results

2 agents

AGENTNAME	TARGET	COMM	ROUNDED	PCT
Nadia Fouad	5000	2134.567	2134.57	42.7%
Omar Sami	4000	3890.204	3890.20	97.3%

✓ **Expression Nesting:** `ROUND` is nested inside a calculation (division and multiplication happen first inside the parentheses) — functions can wrap any valid arithmetic expression, not just a raw column!

— APPLY SCALAR FUNCTIONS · 15 MINUTES

Hands-On Exercise 1 — Scalar Functions

Apply scalar functions to Courses, Payments, and Students tables

🕒 Hands-On Exercise 1 — 15 Minutes

Click "Reveal Solution" below to verify your SQL

1. Round course pricing

Courses

Write a query on Courses that returns CourseName and Price rounded to the nearest whole number as RoundedPrice.

</> Reveal Solution

2. Days since payment

Payments

Write a query on Payments that returns PaymentID, PaymentDate, and the number of days since payment using DATEDIFF.

</> Reveal Solution

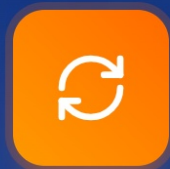
3. Build a contact label

Students

Write a query on Students that returns a FullLabel column combining StudentName and Email using CONCAT.

</> Reveal Solution

Try writing these queries directly in SSMS or Azure Data Studio before revealing the answers!



PART 2 OF 3 • CONVERSION FUNCTIONS

Conversion Functions

Move data safely between types with **CAST**, **CONVERT**, and their **TRY_** variants

— WHY CONVERSION MATTERS

Why Conversion Matters

Every column has a data type — mixing types is where SQL queries quietly break or slow down

Every column has a data type — **mixing types** is where SQL queries quietly break, yield false results, or fail silently!

⚠ Implicit Conversion (Automatic)

SQL Server converts data types automatically, without being asked.

```
-- Works, but conversion is hidden
SELECT '100' + 50; -- returns 150
```

⚠ **Risky:** May cause index scans, incorrect sorting, or unexpected conversion errors on dirty strings!

✅ Explicit Conversion (Controlled)

You control the conversion using CAST or CONVERT — clear and predictable.

```
-- Intent is explicit and reviewable
SELECT CAST('100' AS INT) + 50;
```

✅ **Best Practice:** High performance, zero ambiguity, clear to peer analysts and maintainers!

WHY IT MATTERS FOR A DATA ANALYST:

- Dates stored as text ('20260115') can't be filtered or sorted correctly until converted to a real date type.
- Comparing a VARCHAR column to a number can silently fail, or silently succeed with the wrong result.
- Reports and dashboards expect consistent formats — conversion functions produce that consistency.

— CAST — THE ANSI-STANDARD WAY

CAST — The ANSI-Standard Way

The most portable way to explicitly change a value's data type across relational database engines

SYNTAX: `CAST ( expression AS data_type )`

ANSI-SQL PORTABLE

</> PAYMENTS TABLE TYPE TRANSFORMATIONS

```
SELECT
    Amount,
    CAST(Amount AS INT)           AS AmountWhole,
    CAST(PaymentDate AS VARCHAR(20)) AS PaymentDateText,
    CAST(Amount AS DECIMAL(10,2)) AS AmountFormatted
FROM Payments;
```



Use CAST when: you want code that works the same across SQL Server, PostgreSQL, MySQL and other database systems. It has no formatting options — just a clean type change.

— CONVERT — THE SQL SERVER WAY

CONVERT — The SQL Server Way

Same idea as CAST, plus a style code that controls date and time display formatting

SYNTAX: `CONVERT ( data_type, expression [, style] )`

T-SQL SPECIFIC

</> Date Style Transformations

Payments Table

```
SELECT
    PaymentDate,
    CONVERT(VARCHAR, PaymentDate, 103) AS UK_Format,    -- dd/mm/yyyy
    CONVERT(VARCHAR, PaymentDate, 101) AS US_Format,    -- mm/dd/yyyy
    CONVERT(VARCHAR, PaymentDate, 120) AS ISO_Format    -- yyyy-mm-dd
FROM Payments;
```

● Third parameter controls regional date presentation format

STYLE CODE REFERENCE (CLICK TO TEST):

101 US Standard 07/22/2026

103 UK / European 22/07/2026

112 ISO Compact 20260722

120 ODBC Canonical 2026-07-22 09:00:00



CAST vs CONVERT — which to use? Use CAST for straightforward type changes. Use CONVERT specifically when you need a particular date or number display format — that style-code control is CONVERT's one advantage over CAST.

— FORMATTING LEAD DATES

Live Demo — Formatting Lead Dates

Present CreatedDate in two different formats for two different business audiences

</> Audience-Specific Date Formatting

Leads Table

```
SELECT
    LeadName,
    CreatedDate,
    CONVERT(VARCHAR(10), CreatedDate, 103) AS CreatedDateUK,
    CONVERT(VARCHAR(10), CreatedDate, 112) AS CreatedDateCompact
FROM Leads;
```

Conversion Formats Output

2 leads

LEADNAME	CREATEDDATE (NATIVE)	CREATEDDATEUK (STYLE 103)	CREATEDDATECOMPACT (STYLE 112)
Yara Hassan	2026-06-14	14/06/2026	20260614
Sherif Nabil	2026-06-20	20/06/2026	20260620



CreatedDateCompact (style 112): Sorts correctly as plain text because the year comes first (**YYYYMMDD**) — useful for file names and exports.

COMMON PITFALLS — WHEN CONVERSION FAILS

Common Pitfalls — When Conversion Fails

A safer pair of functions that return NULL instead of crashing your query batch

Dirty Data Simulator:

Notes
value:

Present

"Present"

"100"

⊗ RISKY — THIS CAN ERROR

```
-- Fails if Notes has any
-- non-numeric text in it
SELECT CAST(Notes AS INT)
FROM Attendance;
```

✱ Msg 245: Conversion failed when converting varchar to int.

✓ SAFE — RETURNS NULL INSTEAD

```
-- Same idea, no crash
SELECT TRY_CAST(Notes AS INT)
       AS NotesAsNumber
FROM Attendance;
```

Handled Safely: NULL (query continues without crashing)



TRY_CAST and TRY_CONVERT: Work exactly like CAST and CONVERT, but instead of stopping the whole query with an error, an unconvertible value simply becomes NULL. This is essential when you don't fully trust the source data — free-text fields, imported spreadsheets, or manual entry columns like Attendance.Notes.

— DIAGNOSE & FIX TYPE MISMATCHES · 15 MINUTES

Hands-On Exercise 2 — Type Mismatch Diagnosis

Diagnose and fix two type-mismatch problems on Payments and Leads

🕒 Hands-On Exercise 2 — Diagnose & Fix (15 Minutes)

Click to reveal the correct SQL diagnostics

❌ BROKEN QUERY 1

Payments

```
SELECT PaymentID, Amount
FROM Payments
WHERE PaymentDate = 20260115;
```

Task: Rewrite the WHERE clause so PaymentDate is compared correctly using CAST or CONVERT.

Reveal Fix

⚠️ BROKEN QUERY 2

Leads

```
SELECT LeadName,
       CreatedDate + 30 AS FollowUpDate
FROM Leads;
```

Task: This runs, but adds 30 to the wrong thing. Replace it with the correct date function from this session.

Reveal Fix

❓ Discussion Questions:

Show discussion

Why does Query 1 sometimes appear to work, and sometimes silently return zero rows?



PART 3 OF 3 • CASE WHEN

CASE WHEN

Build **IF-style decision logic** directly inside a SQL query

— WHAT IS CASE WHEN?

What is CASE WHEN?

SQL's version of an IF statement — evaluated row by row from top to bottom, stopping at first match

SQL's version of an **IF statement** — evaluated once for every row.

DECISION FLOW FOR ONE ROW:

Test Price: **250**

Row Value
Price = 250

WHEN Price < 100?
NO → Continue

WHEN Price < 300?
YES → MATCH!

Output Value
'Standard'



Execution Order: CASE WHEN checks conditions in order, top to bottom, and stops at the first one that is TRUE. If nothing matches, it falls through to ELSE (or NULL if there is no ELSE). It behaves like Excel's nested IF() or IFS() — but lives inside the SELECT list, so it runs once per row automatically.

— SIMPLE CASE — MATCHING EXACT VALUES

Simple CASE — Matching Exact Values

Compares a single column against a fixed list of exact discrete values

SYNTAX: `CASE column WHEN value1 THEN result1 ... ELSE default END`

EXACT VALUE MATCH

</> STUDENTS STATUS MAPPING

```
SELECT
  StudentName,
  Status,
  CASE Status
    WHEN 'Active'   THEN 'Currently Studying'
    WHEN 'Completed' THEN 'Graduated'
    WHEN 'Dropped'  THEN 'Withdrawn'
    ELSE 'Unknown'
  END AS StatusLabel
FROM Students;
```



Best for: Exact matches on a single column, like status codes or category labels.

— SEARCHED CASE — MATCHING CONDITIONS

Searched CASE — Matching Conditions

Evaluates full boolean expressions on each line — ranges, inequalities, and multi-column logic

SYNTAX: CASE WHEN cond1 THEN result1 WHEN cond2 THEN result2 ELSE default END

CONDITIONAL PREDICATES

```
</> COURSES PRICING TIER CLASSIFICATION
```

```
SELECT
  CourseName,
  Price,
  CASE
    WHEN Price < 100 THEN 'Budget'
    WHEN Price < 300 THEN 'Standard'
    ELSE 'Premium'
  END AS PriceTier
FROM Courses;
```

✓ **Best for:** Ranges, comparisons (<, >, BETWEEN), or logic spanning more than one column. Most real-world CASE statements are searched CASE.

STUDENT STATUS BREAKDOWN

Live Demo — Student Status Breakdown

Turn raw status codes into friendly labels, then count students in each computed category

</> Grouping by Computed CASE

Students Table

```
SELECT
    CASE Status
        WHEN 'Active' THEN 'Currently Studying'
        WHEN 'Completed' THEN 'Graduated'
        WHEN 'Dropped' THEN 'Withdrawn'
        ELSE 'Unknown'
    END AS StatusLabel,
    COUNT(*) AS StudentCount
FROM Students
GROUP BY
    CASE Status
        WHEN 'Active' THEN 'Currently Studying'
        WHEN 'Completed' THEN 'Graduated'
        WHEN 'Dropped' THEN 'Withdrawn'
        ELSE 'Unknown'
    END;
```

● Computed expression used in both SELECT and GROUP BY

Aggregated Groups

2 rows

STATUSLABEL	STUDENTCOUNT
Currently Studying	184
Graduated	96

⚠ Note the repeat:

The same CASE expression appears in both SELECT and GROUP BY — SQL Server groups by the calculated label, not the original column.

COURSE PRICING TIERS

Live Demo — Course Pricing Tiers

Classic bucketing pattern: turning continuous prices into readable groups for reporting

</> Continuous to Discrete Bucketing

Courses Table

```
SELECT
  CASE
    WHEN Price < 100 THEN 'Budget'
    WHEN Price < 300 THEN 'Standard'
    ELSE 'Premium'
  END AS PriceTier,
  COUNT(*) AS CourseCount,
  AVG(Price) AS AvgPrice
FROM Courses
GROUP BY
  CASE
    WHEN Price < 100 THEN 'Budget'
    WHEN Price < 300 THEN 'Standard'
    ELSE 'Premium'
  END;
```

● Groups numeric ranges into analytical categories

Aggregated Tiers

3 tiers

PRICETIER	COUNT	AVGPRICE
Budget	12	72.40
Standard	21	198.10
Premium	7	415.60



Classic "bucketing" pattern: Turning a continuous number into readable groups for a chart or dashboard filter.

— LEAD FUNNEL STAGE

Live Demo — Lead Funnel Stage

Map text statuses onto numeric funnel sequence (1, 2, 3, 4) for logical dashboard ordering

</> Mapping Text Statuses to Funnel Order

Leads Table

```
SELECT
  Status,
  CASE Status
    WHEN 'New'      THEN 1
    WHEN 'Contacted' THEN 2
    WHEN 'Qualified' THEN 3
    WHEN 'Converted' THEN 4
    ELSE 0
  END AS FunnelStage,
  COUNT(*) AS LeadCount
FROM Leads
GROUP BY Status;
```

● Converts unordered text into an ordered integer sequence

Funnel Output

4 stages

STATUS	FUNNELSTAGE	LEADCOUNT
New	1	140
Contacted	2	95
Qualified	3	48
Converted	4	22



FunnelStage: Lets you ORDER BY a logical sequence instead of alphabetically.

— COMBINE EVERYTHING ON SALES AGENTS · 20 MINUTES

Hands-On Exercise 3 — Sales Agent Classification

Combine CASE WHEN, GROUP BY, and CAST on the SalesAgents dataset

🕒 Hands-On Exercise 3 — Combine Everything (20 Minutes)

SalesAgents Table

1. Classify Commission

Write a CASE WHEN on SalesAgents: below 1000 = 'Low', 1000–3000 = 'Medium', above 3000 = 'High'.
Name column **PerformanceTier**.

Reveal Solution

2. Count Agents Per Tier

Extend the query with **GROUP BY** and **COUNT(*)** to return one row per PerformanceTier with agent count.

Reveal Solution

🌟 STRETCH GOAL

Also CAST MonthlyTarget to **DECIMAL(10,2)** and show Commission as a percentage of target rounded to 1 decimal place.

Reveal Solution

Session Recap & Knowledge Check

Three function families — one shared skill: shaping and enriching data inside the query itself

SCALAR FUNCTIONS

UPPER • TRIM • DATEDIFF • ROUND

CONVERSION FUNCTIONS

CAST • CONVERT • TRY_CAST

CASE WHEN

Simple CASE • Searched CASE

QUICK KNOWLEDGE CHECK (CLICK TO REVEAL ANSWER):

1. A scalar function runs once per _____, while an aggregate function collapses many rows into _____.

Reveal

2. Which function should you use if a text column might contain values that fail to convert cleanly?

Reveal

3. In a searched CASE, what happens if a row matches no WHEN condition and there is no ELSE?

Reveal

4. Name one column from today's tables you would apply DATEDIFF to, and explain why.

Reveal

Core Takeaway: Three function families — one shared skill: shaping data inside the query itself.

INSTANT

✓ SESSION 26 COMPLETED SUCCESSFULLY

THANK YOU

Thank you for your active participation!

Next Session: Multi-Table Relational Joins (INNER, LEFT, RIGHT, FULL OUTER)

[↺ Restart Presentation](#)[Explore Joins & Relations →](#)